

Aprendizaje Automático

Detección de anomalías

Distancias, densidad, PCA e Isolation Forest

Pedro Luiz Ramos

Diplomado en Ciencia de Datos

Anomalía, error y dato poco frecuente

Una **anomalía** se aparta del comportamiento esperado bajo una referencia.

Caso	Qué revisar
Estatura registrada como 1800 cm	Unidad o error de digitación.
Compra muy superior a las habituales	Puede ser válida; investigar su contexto.
Temperatura alta en un equipo	Comparar con su modo de operación.

Una alerta indica qué revisar. No demuestra que el dato esté incorrecto.

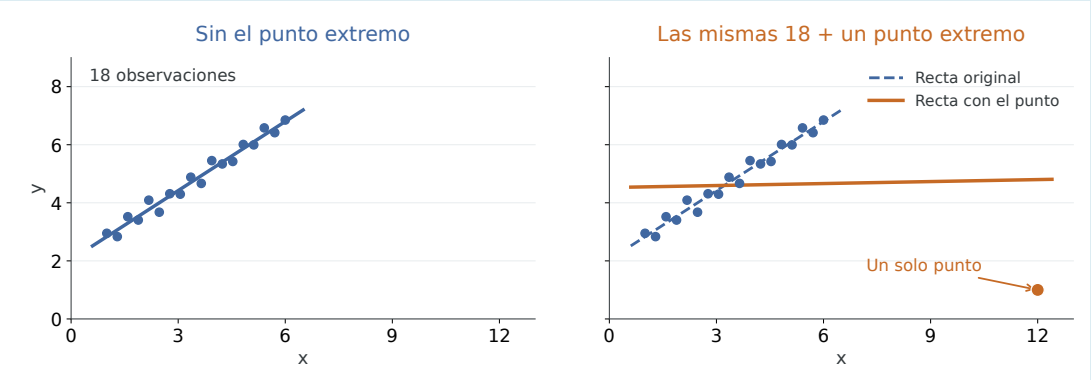
Anomalías globales, contextuales y conjuntas

- **Global**: un monto muy alejado de todos los demás.
- **Contextual**: consumo habitual de día, pero inusual de madrugada.
- **Multivariada**: valores individuales comunes, pero una combinación poco frecuente.

Un equipo puede tener temperatura y presión dentro de sus rangos individuales, pero una relación entre ellas que no corresponde a su operación habitual.

La referencia puede ser toda la muestra, los vecinos o un proceso conocido.

Un solo punto puede cambiar la recta



Datos simulados. Solo se agrega el punto naranja; las otras 18 observaciones no cambian.

Una sola observación puede cambiar mucho la pendiente de la recta.

Revisar antes de eliminar

1. Verificar unidades, duplicados, faltantes y reglas de registro.
2. Consultar el contexto de los casos marcados.
3. Distinguir un error corregible de un evento real poco frecuente.
4. Comparar resultados con y sin el caso si afecta un análisis.

En fraude o fallas, los casos poco frecuentes pueden ser el objetivo del estudio. En otros análisis, interesa reducir su influencia sin perder información.

No reemplazamos automáticamente cada valor marcado por la media.

Influencia sobre medias y modelos

Datos: $1, 2, 3$. La media es

$$\bar{x} = \frac{1+2+3}{3} = 2.$$

Al incorporar 30:

$$\bar{x} = \frac{1+2+3+30}{4} = 9.$$

En regresión y K-means se minimizan sumas de cuadrados:

$$\text{residuo } 2 \Rightarrow 2^2 = 4, \qquad \text{residuo } 20 \Rightarrow 20^2 = 400.$$

Un caso puede influir mucho, aunque sea solo una observación.

Regla del rango intercuartílico

El rango intercuartílico es $RIC = Q_3 - Q_1$. Se marcan valores fuera del intervalo

$$[Q_1 - 1.5\,RIC,\ Q_3 + 1.5\,RIC]\,.$$

- Usa el centro del 50 % de los datos, no la media y desviación.
- No requiere una distribución normal.
- En variables muy asimétricas puede marcar valores válidos de la cola.

Es una regla de exploración, no una prueba automática de error.

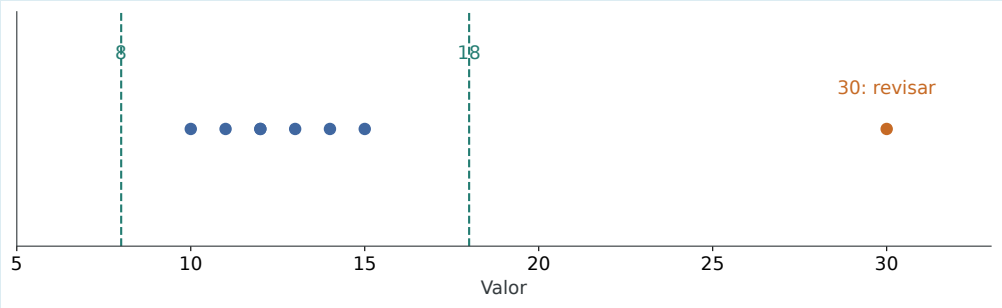
RIC: ejemplo resuelto

Datos: $10, 11, 12, 12, 13, 14, 15, 30$. Con cuartiles interpolados como en los códigos:

$$Q_1 = 11.75, \quad Q_3 = 14.25, \quad RIC = 2.5.$$

$$L = 11.75 - 1.5(2.5) = 8, \qquad U = 14.25 + 1.5(2.5) = 18.$$

Como $30 > 18$, se marca el valor 30.



Puntaje z y sensibilidad a los extremos

$$z_i = \frac{x_i - \bar{x}}{s}.$$

En los mismos ocho valores:

$$\bar{x} = 14.625, \quad s \approx 6.413, \quad z_{30} = \frac{30 - 14.625}{6.413} \approx 2.398.$$

RIC marca 30, pero el corte $|z| > 3$ no lo marca. El propio extremo elevó la media y la desviación. Bajo una normal: aproximadamente 4.55 % queda fuera de $\pm 2\sigma$ y 0.27 % fuera de $\pm 3\sigma$.

El umbral depende del objetivo; ninguna de estas reglas es universal.

RIC en R y Python

R

```r
valores <- c(10,11,12,12,13,14,15,30)
cuartiles <- quantile(valores,
                      c(0.25, 0.75))
ric <- diff(cuartiles)
limite_inferior <- cuartiles[1] - 1.5*ric
limite_superior <- cuartiles[2] + 1.5*ric
alerta_ric <- valores < limite_inferior |
              valores > limite_superior
```

Python

```python
valores = np.array
    ([10,11,12,12,13,14,15,30])
q1, q3 = np.quantile(valores, [.25, .75])
ric = q3 - q1
limite_inferior = q1 - 1.5*ric
limite_superior = q3 + 1.5*ric
alerta_ric = ((valores < limite_inferior) |
              (valores > limite_superior))
```

Se conserva el dato y se agrega una marca para su revisión.

Escala y datos del ejemplo multivariado

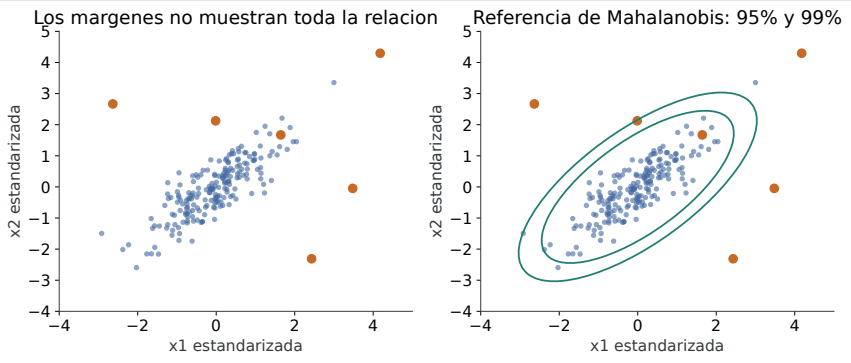
`puntos.csv`: **200 observaciones simuladas correlacionadas y 6 casos adicionales**. El identificador no participa en las distancias.

$$z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}.$$

- Una variable en pesos y otra en metros no son directamente comparables.
- Estandarizar fija una escala común; no elimina atípicos ni correlaciones.
- R y Python usan desviación muestral para obtener la misma escala.

En este ejemplo exploramos una muestra. Más adelante separamos entrenamiento y prueba.

Anomalías en la relación entre variables



Las elipses consideran la dirección y variabilidad de la nube. Un punto puede tener valores marginales moderados y quedar lejos de esa relación.

Distancia de Mahalanobis

$$D_i^2 = (z_i - \bar{z})^\top S^{-1}(z_i - \bar{z}),$$

donde S es la matriz de covarianza.

- Ajusta la distancia por varianzas y correlaciones.
- Para una referencia aproximadamente normal con p variables, un corte orientativo es $D_i^2 > \chi^2_{p,0.99}$.
- En dos variables, $\chi^2_{2,0.99} \approx 9.210$.

Es una aproximación con parámetros estimados, no una garantía de 1 % de falsas alertas.

Mahalanobis: cuenta con dos variables

Referencia hipotética: media $(0, 0)$, varianzas 1 y correlación 0.9.

$$S = \begin{pmatrix} 1 & 0.9 \\ 0.9 & 1 \end{pmatrix}, \qquad S^{-1} = \frac{1}{0.19} \begin{pmatrix} 1 & -0.9 \\ -0.9 & 1 \end{pmatrix}.$$

Punto	Distancia euclidiana al cuadrado	D^2
$(1, 1)$	$1^2 + 1^2 = 2$	$(1 + 1 - 1.8)/0.19 = 1.053$
$(1, -1)$	$1^2 + (-1)^2 = 2$	$(1 + 1 + 1.8)/0.19 = 20$

El segundo punto contradice la correlación positiva de la referencia.

Mahalanobis en R y Python

R

```r
distancia_m <- mahalanobis(z,
  center = colMeans(z), cov = cov(z))
umbral_m <- qchisq(0.99, df = ncol(z))
alerta_m <- distancia_m > umbral_m
```

Python

```python
inversa = np.linalg.inv(
    np.cov(z, rowvar=False))
diferencia = z - z.mean(axis=0)
distancia_m = np.sum(
    (diferencia @ inversa)*diferencia,
    axis=1)
umbral_m = chi2.ppf(.99, df=z.shape[1])
alerta_m = distancia_m > umbral_m
```

`mahalanobis()` devuelve D^2, no D. La covarianza debe ser invertible. Extremos fuertes pueden distorsionarla; una alternativa es estimarla de manera robusta.

kNN para detectar puntos aislados

Se calcula la distancia a los k vecinos más cercanos, **excluyendo el propio punto**. Aquí usamos como puntaje la distancia al vecino k:

$$\text{score}_i = d\big(i, \text{vecino } k\big).$$

Ejemplo con $k = 3$:

Punto	Distancias a los tres vecinos	Puntaje
A	$0.2, 0.3, 0.5$	0.5
B	$2.0, 2.3, 2.8$	2.8

k es el número de vecinos; no es un número de clusters.

kNN en R y Python

R

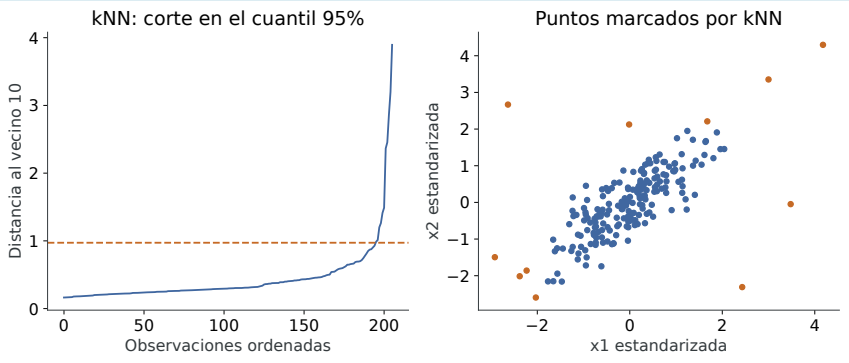
```r
k <- 10
puntaje_knn <- kNNdist(z, k = k)
umbral_knn <- quantile(puntaje_knn, .95)
alerta_knn <- puntaje_knn > umbral_knn
```

Python

```python
k = 10
vecinos = NearestNeighbors(
    n_neighbors=k).fit(z)
distancias, indices = vecinos.kneighbors()
puntaje_knn = distancias[:, -1]
umbral_knn = np.quantile(puntaje_knn, .95)
alerta_knn = puntaje_knn > umbral_knn
```

En Python, llamar `kneighbors()` sin pasar `X` excluye al propio punto de entrenamiento. La **última columna** es la distancia al vecino 10.

kNN: ranking y corte



Se marcan **11 de 206 observaciones** por superar el cuantil 95 %.

El cuantil fija una cantidad aproximada de alertas, no confirma que esas filas sean errores.

Local Outlier Factor: densidad local

LOF compara la densidad alrededor de un punto con la de sus vecinos.

$$\text{reach}_k(i,j) = \text{máx}\{d(i,j),\ k\text{-dist}(j)\},$$

$$\text{lrd}_k(i) = \frac{1}{\text{promedio de } \text{reach}_k(i,j)}, \qquad \text{LOF}_k(i) = \frac{\text{promedio de lrd de sus vecinos}}{\text{lrd}_k(i)}.$$

- LOF cerca de 1: densidad comparable a la de sus vecinos.
- LOF bastante mayor que 1: menor densidad local, candidato a revisión.

LOF no necesita que toda la muestra tenga la misma densidad.

LOF: ejemplo numérico resuelto

Para un punto A, las distancias de accesibilidad a tres vecinos son $0.4, 0.5, 0.6$. Entonces:

$$\text{promedio} = \frac{0.4 + 0.5 + 0.6}{3} = 0.5, \qquad \text{lrd}(A) = \frac{1}{0.5} = 2.$$

Supongamos que las densidades locales calculadas de sus vecinos son $6, 5, 7$:

$$\text{densidad media de vecinos} = \frac{6 + 5 + 7}{3} = 6, \qquad \text{LOF}(A) = \frac{6}{2} = 3.$$

A tiene menor densidad que su entorno. El valor 3 no es una probabilidad.

LOF en R y Python

R

```
puntaje_lof <- lof(z, minPts = k + 1)
umbral_lof <- 1.5
alerta_lof <- puntaje_lof > umbral_lof
```

Python

```
modelo_lof = LocalOutlierFactor(
    n_neighbors=k)
modelo_lof.fit_predict(z)
puntaje_lof = (
    -modelo_lof.negative_outlier_factor_)
alerta_lof = puntaje_lof > 1.5
```

- R: `minPts` incluye al propio punto; usamos $k + 1$.
- Python guarda el **opuesto de LOF**; cambiamos el signo.
- El corte 1.5 es ilustrativo. Se revisa su sensibilidad al número de vecinos.

DBSCAN: núcleo, borde y ruido

DBSCAN forma grupos conectando regiones densas. No requiere elegir k.

- ε: radio para considerar cercanos dos puntos.
- `minPts`: cantidad mínima dentro del radio, **incluyendo al propio punto**.

Con `minPts`=6:

Punto	Condición	Tipo
A	Tiene al menos 6 puntos en su radio	Núcleo
B	Tiene menos de 6, pero está cerca de un núcleo	Borde
C	No es núcleo ni está cerca de un núcleo	Ruido

Un punto de borde pertenece a un grupo. No se marca como ruido.

DBSCAN en R y Python

R

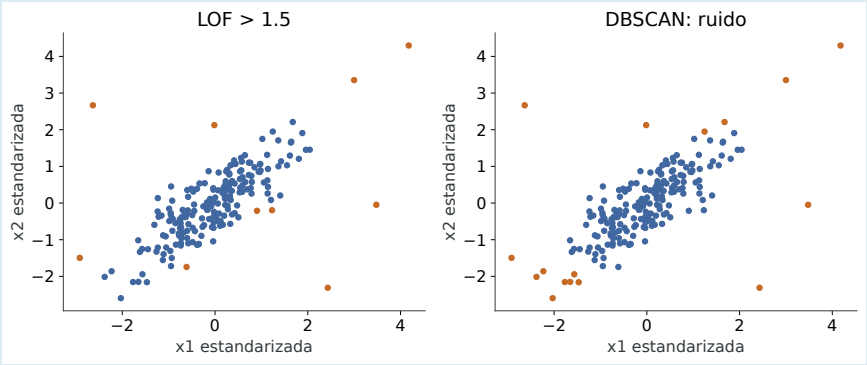
```r
epsilon <- 0.45
min_puntos <- 6
kNNdistplot(z, k = min_puntos - 1)
abline(h = epsilon, col = "red")
modelo_db <- dbscan(z, eps = epsilon,
                    minPts = min_puntos)
alerta_db <- modelo_db$cluster == 0
```

Python

```python
epsilon = 0.45
min_puntos = 6
modelo_db = DBSCAN(
    eps=epsilon,
    min_samples=min_puntos).fit(z)
alerta_db = modelo_db.labels_ == -1
```

Ruido: **0 en R** y **-1 en Python**. El gráfico del vecino 5 orienta la elección de ε; también se revisan soluciones cercanas. No se importa un ε de otra escala.

LOF y DBSCAN sobre los mismos puntos



LOF marca **10 puntos**; DBSCAN deja **16 como ruido**.

Son criterios distintos: densidad relativa a vecinos y pertenencia a regiones densas.

Coincidencias y diferencias entre detectores

Casos adicionales: identificadores 201 a 206.

ID	Mahalanobis	kNN	LOF	DBSCAN
201	Sí	Sí	Sí	Sí
202	Sí	Sí	Sí	Sí
203	Sí	Sí	Sí	Sí
204	No	Sí	Sí	Sí
205	Sí	Sí	Sí	Sí
206	No	No	No	No

El caso 206 es $(1.9, 1.9)$: sigue la relación de la nube y no se marca.

Agregar un caso para explorar no lo convierte necesariamente en anómalo.

PCA: proyección y reconstrucción

Sea Z la matriz de variables centradas y estandarizadas, y P_r las cargas de las primeras r componentes.

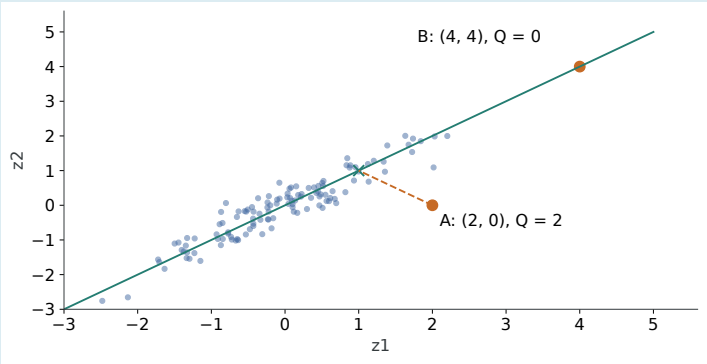
$$T_r = ZP_r, \qquad \widehat{Z} = T_r P_r^\top, \qquad E = Z - \widehat{Z}.$$

- T_r contiene las coordenadas en el subespacio retenido.
- $\widehat{Z}$ reconstruye cada observación usando ese subespacio.
- La diferencia mide lo que no quedó representado.

$$Q_i = \sum_{j=1}^{p} (z_{ij} - \widehat{z}_{ij})^2.$$

Comparamos patrones entre variables, no solo valores grandes o pequeños.

PCA: distancia al subespacio retenido



La recta representa una componente. A se aparta de ella; B está lejos del centro, pero permanece sobre la recta.

PCA: reconstrucción paso a paso

Una componente con carga $p_1 = (1,1)^\top/\sqrt{2}$. Para $z = (2,0)$:

$$t_1 = zp_1 = \frac{2+0}{\sqrt{2}} = \sqrt{2}.$$

$$\widehat{z} = t_1 p_1^\top = \sqrt{2}\frac{(1,1)}{\sqrt{2}} = (1,1).$$

$$e = z - \widehat{z} = (2,0) - (1,1) = (1,-1), \qquad Q = 1^2 + (-1)^2 = 2.$$

El puntaje Q suma residuos al cuadrado en la escala usada para ajustar.

Lo que Q no detecta por sí solo

Para $z = (4, 4)$, usando la misma componente:

$$t_1 = 4\sqrt{2}, \quad \widehat{z} = (4, 4), \quad Q = 0.$$

El punto puede ser extremo **dentro** del subespacio sin error de reconstrucción. Una medida complementaria es

$$T_i^2 = \sum_{\ell=1}^{r} \frac{t_{i\ell}^2}{\lambda_\ell}.$$

- Q: desviación fuera del subespacio retenido.
- T^2: posición extrema dentro del subespacio.

Con todas las componentes, Q es prácticamente cero para cualquier punto.

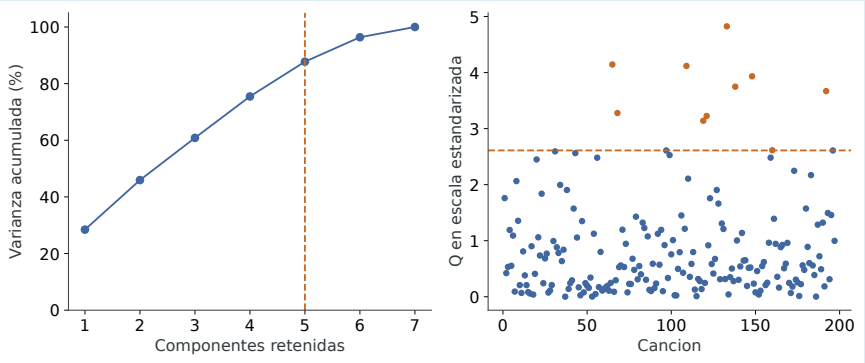
Canciones: siete características numéricas

`canciones.csv`: **197 canciones**, tomadas del ejemplo del aula anterior.

Variable	Característica
`acousticness`	Carácter acústico
`danceability`	Bailabilidad
`energy`	Energía
`liveness`	Presencia de público / grabación en vivo
`loudness`	Nivel de sonoridad, en dB
`speechiness`	Presencia de voz hablada
`instrumentalness`	Carácter instrumental

`id` identifica la fila; queda fuera del PCA. No hay etiquetas de anomalía.

Canciones: varianza y error de reconstrucción



Las primeras **5 componentes explican 87.75 %** de la varianza. El corte de Q en el cuantil 95 % marca **10 canciones para revisar**.

Explicar mucha varianza no garantiza detectar todas las anomalías.

PCA y Q en R

```
canciones <- read.csv("canciones.csv")
x_canciones <- as.matrix(canciones[, -1])
z_canciones <- scale(x_canciones)
pca_canciones <- prcomp(z_canciones, center = FALSE, scale. = FALSE)
r <- 5
proyeccion <- pca_canciones$x[, 1:r]
reconstruccion <- proyeccion %*% t(pca_canciones$rotation[, 1:r])
q_canciones <- rowSums((z_canciones - reconstruccion)^2)
umbral_canciones <- quantile(q_canciones, 0.95)
alerta_canciones <- q_canciones > umbral_canciones
```

`rowSums()` implementa la suma que define Q.

No mezclamos residuos de dB con residuos de variables entre 0 y 1.

PCA y Q en Python

```
canciones = pd.read_csv("canciones.csv")
x_canciones = canciones.drop(columns="id")
z_canciones = (x_canciones - x_canciones.mean()) / x_canciones.std(ddof=1)
pca_canciones = PCA().fit(z_canciones)
r = 5
proyeccion = pca_canciones.transform(z_canciones)[:, :r]
reconstruccion = (proyeccion @ pca_canciones.components_[:r]
                  + pca_canciones.mean_)
q_canciones = np.sum((z_canciones.to_numpy() - reconstruccion)**2, axis=1)
umbral_canciones = np.quantile(q_canciones, 0.95)
alerta_canciones = q_canciones > umbral_canciones
```

Se reconstruye con las primeras cinco componentes, igual que en R.

Canciones con mayor Q

Identificador	Q
133	4.825
65	4.144
109	4.118
148	3.934
138	3.749

Revisamos los perfiles originales de esas canciones y sus residuos. Una combinación musical poco habitual puede ser real y válida.

Sin etiquetas no calculamos acuracia, sensibilidad ni AUC. Este ranking es exploratorio.

Monitorear observaciones nuevas con PCA

1. Ajustar medias, escalas y componentes con datos de referencia.
2. Establecer el corte con esa referencia o con validación separada.
3. Estandarizar cada nuevo dato con las **mismas** medias y escalas.
4. Proyectar, reconstruir y comparar Q con el corte fijado.
5. Investigar las alertas y revisar cambios persistentes del proceso.

No actualizamos el modelo incorporando automáticamente los datos que generaron alarmas.

Si cambia la operación normal, el reajuste debe ser una decisión documentada.

Isolation Forest: aislar mediante cortes aleatorios

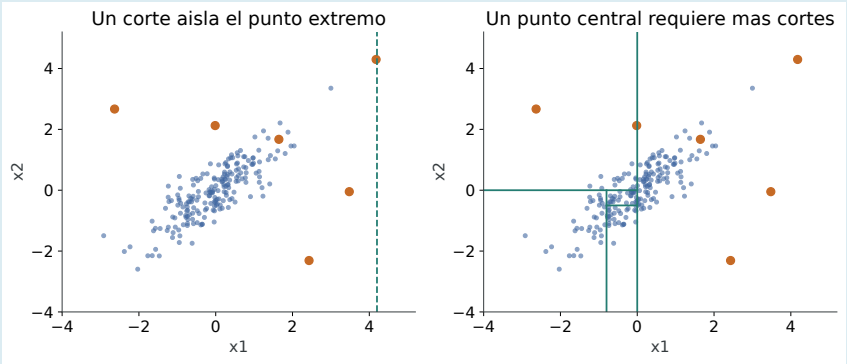
Cada **árbol de aislamiento** usa una submuestra de los datos. En cada nodo:

1. Elegir una variable al azar.
2. Elegir un corte al azar dentro de su rango.
3. Separar los puntos a ambos lados y repetir.

Se detiene al aislar puntos o alcanzar el límite de profundidad. Las hojas con varios puntos requieren una corrección de longitud.

Los puntos fáciles de aislar suelen tener caminos más cortos. No se usan clases para construir los cortes.

Isolation Forest: ejemplo de aislamiento



Un corte puede separar el punto extremo. Aislar uno del centro suele requerir más divisiones. El resultado del bosque promedia muchos **caminos aleatorios**.

Puntaje de Isolation Forest

Una forma clásica del puntaje es

$$s(x) = 2^{-E[h(x)]/c(\psi)},$$

donde $E[h(x)]$ es la longitud media corregida del camino y $c(\psi)$ normaliza por el tamaño de la submuestra. Ejemplo hipotético con $c(\psi) = 8$:

$$h = 2 \Rightarrow s = 2^{-2/8} \approx 0.841, \qquad h = 8 \Rightarrow s = 2^{-8/8} = 0.5.$$

Un puntaje alto indica aislamiento fácil. No significa 84.1 % de probabilidad de anomalía.

Árboles, submuestra y cantidad de alertas

Parámetro	Qué controla
300 **árboles**	Cuántos caminos se combinan para obtener el puntaje.
Submuestra de **256**	Cuántos registros usa cada **árbol**; no son 256 variables.
Contamination	En sklearn, determina el corte al convertir puntajes en clases.
Cuantil **95 %**	En esta práctica fijamos un corte propio con entrenamiento.

La fracción esperada de alertas no es la fracción de datos utilizada en cada árbol.

Cardio: evaluar con etiquetas disponibles

`cardio.csv` proviene de `cardio.mat`, usado en el aula anterior.

- **1831 registros y 21 variables numéricas** de un benchmark.
- Etiquetas disponibles: **1655 normales y 176 anomalías**.
- 0 = normal; 1 = anomalía.
- Las variables se nombran `v1`,...,`v21`; no se reinterpretan como diagnósticos.

$$176/1831 \approx 9.61\,\%.$$

Es una práctica de detección sobre un benchmark, no una herramienta clínica validada.

Entrenamiento y prueba: 70 % y 30 %

La partición aleatoria queda guardada en `conjunto` para usar las mismas filas en R y Python.

Conjunto	Normales	Anomalías	Total
Entrenamiento	1159	122	1281
Prueba	496	54	550

- Ajustamos escala, componentes, bosque y umbrales con entrenamiento.
- `id`, `conjunto` y `anomalia` quedan fuera de los predictores.
- Entrenamiento no se filtra por etiqueta: contiene casos normales y anómalos.

Las etiquetas de prueba se consultan al evaluar, no al ajustar ni fijar cortes.

PCA con Cardio: ajuste y corte

1. Estandarizar usando medias y desviaciones de entrenamiento.
2. Retener las primeras componentes hasta explicar al menos 80 %.
3. Reconstruir entrenamiento y calcular sus Q.
4. Fijar u_Q en el cuantil 95 % de esos Q.
5. Aplicar escala, cargas y u_Q sin cambios a prueba.

En esta partición se retienen **8 componentes** y $u_Q \approx$ **11.990**.

$$\widehat{y}_i = \mathbf{1}(Q_i > 11.990).$$

El 80 % y 95 % son decisiones docentes fijadas antes de revisar prueba.

Isolation Forest en R

```
modelo_if <- isolation.forest(z_train, ndim = 1, ntrees = 300,
                              sample_size = 256, seed = 2026, nthreads = 1)
score_train <- predict(modelo_if, z_train, type = "score")
score_test <- predict(modelo_if, z_test, type = "score")
umbral_if <- quantile(score_train, 0.95)
clase_if <- as.integer(score_test > umbral_if)
```

- `ndim=1`: un corte sobre una variable en cada nodo.
- `isotree` entrega puntajes mayores para mayor anomalidad.
- El corte se obtiene de entrenamiento, no de los puntajes de prueba.

La salida es una marca: 1 = anomalía; 0 = normal.

Isolation Forest en Python

```
modelo_if = IsolationForest(n_estimators=300, max_samples=256,
                            contamination="auto", random_state=2026, n_jobs=1)
modelo_if.fit(z_train)
score_train = -modelo_if.score_samples(z_train)
score_test = -modelo_if.score_samples(z_test)
umbral_if = np.quantile(score_train, 0.95)
clase_if = (score_test > umbral_if).astype(int)
```

- `score_samples()` es mayor para puntos más normales.
- Cambiamos el signo: ahora un puntaje mayor implica mayor anomalidad.
- Usamos nuestro cuantil; no el corte de `predict()`.

Convenciones y orientación de puntajes

Salida	Interpretación	En nuestros códigos
`isotree: score`	Alto = más anómalo	Se usa directamente
`sklearn: score_samples`	Bajo = más anómalo	Se cambia el signo
`sklearn: predict`	-1 anómalo; +1 normal	No usamos ese corte

Si se usara `predict()`, la conversión correcta sería

$$\widehat{y} = \mathbf{1}(\text{predict} = -1),$$

no interpretar +1 como anomalía.

Unificar la dirección permite calcular AUC sin invertir normales y anomalías.

Matriz de confusión: cuenta resuelta

Isolation Forest en Python, corte en el cuantil 95 % de entrenamiento:

	Predicha normal	Predicha anomalía
Real normal	483	13
Real anomalía	38	16

$$\text{Acuracia} = \frac{483 + 16}{550} = 0.9073, \qquad \text{Sensibilidad} = \frac{16}{38 + 16} = 0.2963.$$

$$\text{Precisión de las alertas} = \frac{16}{13 + 16} = 0.5517.$$

Detectó 16 de 54 anomalías; 13 de sus 29 alertas eran casos normales.

PCA e Isolation Forest en prueba

Versión	Método	Acuracia	Sensib.	Precisión	AUC
R / Python	PCA	0.8800	0.2963	0.3636	0.8503
R	Isolation Forest	0.9127	0.2963	0.6154	0.9220
Python	Isolation Forest	0.9073	0.2963	0.5517	0.9072

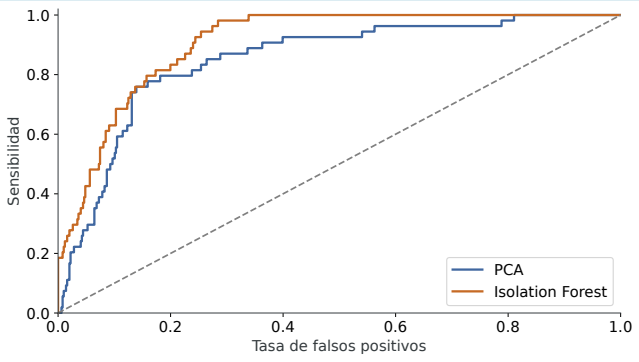
Marcar todos los casos como normales daría

$$\text{Acuracia} = 496/550 = 0.9018, \qquad \text{Sensibilidad} = 0.$$

PCA coincide entre lenguajes. Los bosques difieren por implementación y aleatoriedad; una misma semilla no genera los mismos cortes en ambos programas.

No basta con mirar acuracia ni con elegir el resultado más alto de esta tabla.

AUC y el umbral de alertas



AUC evalúa el **orden de los puntajes** a lo largo de distintos cortes. La sensibilidad y precisión de la tabla corresponden a **un corte concreto**.

Una AUC alta no implica que el cuantil 95 % detecte la mayoría de las anomalías.

Cambiar el umbral: efecto sobre las alertas

Isolation Forest en Python; los tres cortes se calculan con entrenamiento.

Cuantil	Umbral	Alertas en prueba	Sensibilidad
90 %	0.4994	59	$26/54 = 0.4815$
95 %	0.5291	29	$16/54 = 0.2963$
99 %	0.5756	10	$10/54 = 0.1852$

- Corte mayor: menos alertas y, aquí, menor sensibilidad.
- El ranking no cambia; la AUC se mantiene en 0.9072.
- El porcentaje de alertas en prueba no tiene por qué igualar el de entrenamiento.

La tabla ilustra el efecto. Para elegir un corte se usa validación, no prueba.

Extended Isolation Forest

- El Isolation Forest clásico corta sobre una variable: divisiones paralelas a los ejes.
- La extensión permite cortes oblicuos mediante combinaciones de variables.
- Puede reducir artefactos ligados a la orientación de los cortes.

En R, `isotree` permite modelos extendidos con `ndim>1`. El `IsolationForest` estándar de sklearn implementa el modelo clásico.

Es una extensión para explorar; no garantiza mejores resultados en todas las bases.

Elección del método

Método	Qué se considera inusual
RIC / z	Valor extremo de una variable.
Mahalanobis	Distancia a una referencia considerando covarianzas.
kNN	Separación respecto de los vecinos más cercanos.
LOF	Densidad menor que la del entorno local.
DBSCAN	Ausencia de pertenencia a una región densa.
PCA y Q	Combinación mal reconstruida por el subespacio retenido.
Isolation Forest	Facilidad de aislamiento mediante cortes aleatorios.

La elección depende de la estructura de los datos y del tipo de anomalía buscada.

Evaluación con y sin etiquetas

Si hay etiquetas confiables:

- Acuracia, sensibilidad, precisión y falsas alertas para un corte.
- ROC-AUC para el ranking; precisión-recall si las anomalías son muy escasas.
- Separar ajuste, selección del corte y evaluación final.

Si no hay etiquetas:

- Revisar casos con especialistas y comparar perfiles.
- Examinar estabilidad con distintos vecinos, escalas o semillas.
- Considerar cuántas alertas es posible investigar.

No aplicamos log loss ni Brier directamente a puntajes que no son probabilidades calibradas.

Decisiones después de una alerta

1. Identificar el registro y las variables que motivaron su revisión.
2. Confirmar si hubo un error o un cambio real del comportamiento.
3. Registrar la decisión: corregir, conservar, investigar o escalar.
4. Evaluar falsas alertas y anomalías no detectadas cuando haya evidencia.
5. Revisar el modelo si cambió la referencia de operación normal.

La detección organiza la revisión; la decisión final requiere contexto.